MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Conception Détaillée

Programmation du Plakoto

Auteurs :
YA Hassrol
GHANIME Joseph
BENNAMANE Mahdi
REN Ziyi

Encadrant :
BOUZY Bruno

Groupe : L2N1

Table des matières

1 Introduction **2**
1.1 ***Objectif du Document*** 2
1.2 ***Portée*** 2

2 Description des Modules **2**
2.1 ***Interface Textuelle*** 2
2.1.1 **Fonctionnalités** 2
2.1.2 **Structure des Données** 2
2.1.3 **Fonctions Principales** 3
2.1.4 **Gestion des Erreurs et Messages d'Avertissement** 4
2.2 ***Intelligence Artificielle (Monte Carlo)*** 5
2.2.1 **Fonctionnalités** 5
2.2.2 **Structure des Données** 6
2.2.3 **Algorithmes et Fonctions Principales** 6

3 Interfaces et Communication entre Modules **6**

4 Gestion des Erreurs et Sécurité **6**
4.1 ***Stratégies de Gestion des Erreurs*** 6
4.2 ***Sécurité*** 6

5 Technologies et Outils **6**

6 Conclusion **6**

1 Introduction

1.1 Objectif du Document

Ce document fournit une description détaillée de la conception du projet, en se basant sur la conception générale. Il a pour but de guider l'implémentation en définissant précisément les fonctionnalités, les structures de données, les interfaces et les algorithmes utilisés.

1.2 Portée

— Détailler le fonctionnement interne de chaque module.
— Décrire les structures de données et les formats d'échange entre composants.
— Fournir une base claire pour l'implémentation et les tests.

2 Description des Modules

2.1 Interface Textuelle

L'interface textuelle constitue le premier mode d'interaction du joueur avec le jeu. Elle permet aux joueurs de visualiser l'état du plateau et d'effectuer des actions via des commandes textuelles.

2.1.1 Fonctionnalités

L'interface textuelle gère les fonctionnalités suivantes :
— **Affichage du plateau** : présentation du plateau sous forme ASCII avec indication des positions des pions.
— **Gestion des entrées utilisateur** : interprétation des commandes saisies par le joueur pour exécuter les actions de jeu.
— **Lancer de dés** : simulation du lancer de deux dés, avec gestion des doubles.
— **Déplacement des pions** : validation et exécution des mouvements des pions selon les règles du jeu.
— **Vérification de victoire** : contrôle des conditions de fin de partie.
— **Affichage des messages d'erreur et d'information** : indication des erreurs de commande et confirmation des actions réussies.

2.1.2 Structure des Données

L'interface textuelle repose sur les structures de données suivantes :
— `plateau` : dictionnaire où chaque clé est un indice de 0 à 23 représentant les 24 cases du plateau, et chaque valeur est une liste contenant les pions présents sur la case.
— `joueurs` : une liste contenant les identifiants des deux joueurs (`"B"` pour blanc et `"N"` pour noir).
— `dés` : une liste stockant les valeurs des dés actuellement disponibles pour jouer.

2.1.3 Fonctions Principales

Les principales fonctions qui assurent le bon fonctionnement de l'interface textuelle sont :

- `init_plateau()` :
  - Initialise un dictionnaire représentant les 24 cases du plateau.
  - Place 15 pions blancs (`B`) sur la première case et 15 pions noirs (`N`) sur la dernière case.
- `afficher_plateau(plateau)` :
  - Affiche chaque case du plateau avec son contenu (pions `B` et `N`) sous forme ASCII.
  - Format d'affichage : `1 : BBBBBBBBBBBBBB`.
- `lance_de()` :
  - Simule un lancer de deux dés.
  - Si les deux dés ont la même valeur (double), retourne quatre occurrences de cette valeur.
  - Sinon, retourne les deux valeurs obtenues.
- `afficher_commandes()` :
  - Affiche les commandes disponibles pour les joueurs :
    - `help` : afficher les commandes disponibles.
    - `lance` : lancer les dés.
    - `plateau` : afficher l'état du plateau.
    - `move <origine> <destination>` : déplacer un pion.
    - `quit` : quitter le jeu.
- `is_move_legal_by_dice(origine, destination, dice, joueur)` :
  - Vérifie si un déplacement est conforme aux règles et aux valeurs des dés.
  - Blancs (`B`) se déplacent de **droite à gauche** (indice décroissant).
  - Noirs (`N`) se déplacent de **gauche à droite** (indice croissant).
  - Vérifie si la distance entre l'origine et la destination correspond à l'une des valeurs des dés restants.
- `deplacer_pion(plateau, origine, destination, joueur)` :
  - Vérifie si la case d'origine contient bien un pion appartenant au joueur actif.
  - Vérifie si la case de destination est autorisée :
    - Case vide : déplacement autorisé.
    - Case occupée par des pions alliés : déplacement autorisé.
    - Case contenant un seul pion adverse : affichage d'un message de blocage.
    - Case contenant plusieurs pions adverses : déplacement interdit.
  - Effectue le déplacement en retirant le pion de la case d'origine et en l'ajoutant à la case de destination.

    ——————————a refaire ——————————-
- `check_victoire(plateau, joueur)` :
  - Vérifie si un joueur a atteint la condition de victoire :
    - **Blancs gagnent** si tous leurs pions sont sur la première case (`plateau[0]`).
    - **Noirs gagnent** si tous leurs pions sont sur la dernière case (`plateau[23]`).

  ——————————a refaire ——————————-
- `gestion_commande(cmd, plateau, joueur, dice)` :
  - Analyse la commande saisie par l'utilisateur et exécute l'action correspondante.
  - Gère les cas d'erreur (format incorrect, mouvement illégal, dés déjà lancés, etc.).
  - Met à jour l'état du jeu après chaque action réussie.

  - Retourne un indicateur sur la poursuite du jeu et les dés restants.
- `main()` :
  - Boucle principale du jeu.
  - Initialise le plateau et affiche l'état initial.
  - Gère l'alternance entre les joueurs.
  - Vérifie après chaque action si un joueur a gagné.
  - Fournit un retour utilisateur sur chaque action effectuée.

2.1.4 Gestion des Erreurs et Messages d'Avertissement

L'interface textuelle inclut plusieurs messages d'erreur et d'avertissement pour guider l'utilisateur et éviter les actions illégales. Voici une analyse détaillée des cas d'erreur et de leurs causes :

Message d'erreur	Cause possible	Solution proposée
Erreur : la case d'origine est vide.	Le joueur tente de déplacer un pion depuis une case sans pion.	Vérifier que l'origine contient un pion avant d'effectuer un déplacement.
Erreur : la pièce à la case d'origine ne vous appartient pas.	Le joueur tente de déplacer un pion qui appartient à l'adversaire.	Vérifier que le pion sélectionné appartient bien au joueur actif.
Erreur : la case de destination est bloquée par l'adversaire.	La case de destination contient plusieurs pions adverses, rendant le déplacement impossible.	Choisir une destination valide selon les règles de déplacement.
Erreur : ce déplacement ne correspond pas aux valeurs des dés ou à la direction requise.	Le joueur tente de déplacer un pion avec une distance non autorisée par les dés ou dans une direction invalide.	Vérifier que la distance correspond à l'un des dés et que la direction est conforme aux règles.
Erreur : position hors limites. Veuillez entrer un nombre entre 1 et 24.	L'utilisateur a saisi une case hors du plateau.	Vérifier que les positions d'origine et de destination sont bien comprises entre 1 et 24.
Erreur de format de commande. Usage correct : move .	La commande `move` a été saisie avec un format incorrect (ex. : mauvaise syntaxe, trop ou trop peu d'arguments).	Vérifier la syntaxe et s'assurer que la commande suit le bon format.
Vous avez déjà lancé les dés pour ce tour, veuillez directement effectuer un déplacement.	Le joueur tente de lancer les dés alors qu'ils ont déjà été lancés ce tour-ci.	Effectuer un déplacement ou attendre la fin du tour avant de relancer les dés.
Commande inconnue. Veuillez taper 'help' pour afficher la liste des commandes.	L'utilisateur a entré une commande non reconnue.	Vérifier l'orthographe de la commande et consulter `help` pour obtenir la liste des commandes disponibles.
Le joueur X a gagné !	Un joueur a rempli la condition de victoire.	La partie se termine et le jeu affiche le vainqueur.

Ces messages d'erreur permettent d'éviter les comportements indésirables et d'assurer une meilleure expérience utilisateur.

2.2 Intelligence Artificielle (Monte Carlo)

2.2.1 Fonctionnalités

— Simulation des coups possibles pour le joueur IA.
— Calcul des probabilités de victoire selon les stratégies disponibles.
— Sélection du meilleur coup basé sur les évaluations Monte Carlo.

2.2.2 Structure des Données

— `simulation_resultats` : stockage des scores obtenus par chaque coup testé.
— `historique_parties` : enregistrement des décisions et résultats précédents.

2.2.3 Algorithmes et Fonctions Principales

— `simuler_coup(plateau, joueur, depth)` : effectue des simulations aléatoires.
— `evaluer_coup(plateau, joueur)` : attribue un score aux coups possibles.
— `choisir_meilleur_coup(plateau, joueur)` : sélectionne le coup optimal.

3 Interfaces et Communication entre Modules

Module	Entrées	Sorties	Description
Interface Textuelle	Commandes utilisateur	Mise à jour du plateau	Permet de jouer via terminal
Intelligence Artificielle	État du jeu	Coup sélectionné	IA effectue un mouvement
Interface Graphique	Actions utilisateur	Affichage mis à jour	Interaction avec le plateau

——————————a supprimer ——————————-

4 Gestion des Erreurs et Sécurité

4.1 Stratégies de Gestion des Erreurs

— Vérification des entrées utilisateur pour éviter les valeurs invalides.
— Gestion des exceptions pour éviter les crashs.
— Messages d'erreur détaillés pour guider l'utilisateur.

4.2 Sécurité

— Protection contre les entrées malveillantes.
— Tests unitaires sur chaque module pour garantir leur fiabilité.
——————————a supprimer ——————————-

5 Technologies et Outils

Technologie	Utilisation
Python	Langage principal
Pygame	Interface graphique
-	-

6 Conclusion

Ce document fournit toutes les informations nécessaires pour la mise en œuvre détaillée du projet. Chaque module a été défini avec ses fonctionnalités, ses structures de données

et ses interfaces, garantissant une implémentation fluide et organisée. La prochaine étape consiste à démarrer le développement et à valider chaque fonctionnalité via des tests unitaires et des essais pratiques.